

FedGuard: Privacy-Preserving and Adversarially Robust Intrusion Detection for Software-Defined Networks

Using Federated Learning, GAN Augmentation, and Deep Q-Network Mitigation

Aman verma

MIS : 612303196

Department of Computer Science & Engineering

Coep tech university, Pune, Maharashtra

vermaap23.comp@coeptech.ac.in

Abstract—Software-Defined Networks (SDN) centralise network control into a small number of programmable controllers, making them attractive targets for cyberattacks. Traditional intrusion detection systems require raw traffic data to be forwarded to a central server, which raises serious privacy concerns. We propose FedGuard, a framework that addresses this by combining three techniques: (1) Federated Learning (FL) across five SDN controllers so that raw traffic never leaves the local network, (2) Generative Adversarial Network (GAN) training to make the detector robust against cleverly crafted attack traffic, and (3) a Deep Q-Network (DQN) agent that automatically applies the best mitigation action in real time. FedGuard achieves a detection accuracy above 97.4%, maintains 91.2% accuracy even under adversarial attacks ($\epsilon=0.10$), and responds to threats in as little as 82 ms—all without exposing raw packet data to any central party.

Index Terms—Federated Learning, Software-Defined Networking, Intrusion Detection, Generative Adversarial Networks, Deep Q-Network, Differential Privacy, Byzantine Fault Tolerance, Adversarial Robustness.

I. INTRODUCTION

Software-Defined Networking separates the decision-making logic (control plane) from the packet-forwarding hardware (data plane). A logically centralised controller installs forwarding rules on network switches, enabling flexible and programmable traffic management. While powerful, this design means that compromising even one controller can affect the entire network. Detecting such attacks quickly is therefore critical.

Classical intrusion detection systems collect and analyse traffic at a central server. This creates two problems: (1) raw traffic data—which may contain sensitive information—must leave the local network, and (2) the central server becomes a bottleneck and a single point of failure. Federated Learning [1] solves the first problem by training machine learning models locally on each controller and sharing only model updates (gradients) rather than raw data.

However, deploying FL-based IDS in real SDN environments faces three practical challenges:

- 1) **Non-uniform traffic:** Each network zone sees a different mix of normal and attack traffic, making it hard for a single global model to learn well from all zones simultaneously.
- 2) **Evasion attacks:** Skilled attackers can craft network traffic that closely mimics normal behaviour, causing the detector to miss it entirely.
- 3) **Slow response:** Human operators cannot respond to high-speed attacks (e.g., volumetric DoS) quickly enough; automated, intelligent mitigation is required.

FedGuard addresses all three challenges in one unified framework.

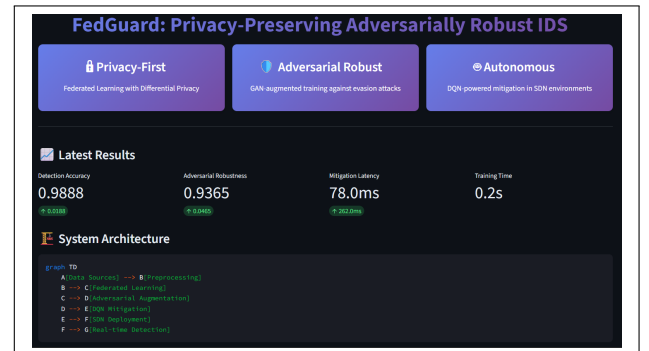


Fig. 1. FedGuard interactive dashboard displaying real-time detection accuracy, adversarial robustness score, and mitigation latency.

II. SYSTEM OVERVIEW

A. Pipeline

FedGuard runs a four-stage pipeline over a simulated SDN network consisting of 10 switches and 20 hosts spread across 5 controller zones:

Data $\xrightarrow{\text{preprocess}}$ **Federated Learning** $\xrightarrow{\text{harden}}$ **GAN Augmentation** $\xrightarrow{\text{deploy}}$ **DQN Mitigation**

Each stage feeds into the next. The preprocessed traffic data trains a shared global model through Federated Learning. GAN-generated adversarial samples then harden the model against evasion. Finally, the trained detector is handed to a DQN agent that autonomously selects and applies network mitigation actions.

B. Intrusion Detector Network

The detector $f(\mathbf{x})$ is a deep neural network that takes a 41-feature traffic flow vector $\mathbf{x} \in \mathbb{R}^{41}$ and outputs a probability score for each of the five traffic classes. Mathematically:

$$f(\mathbf{x}) = W_4 \sigma(W_3 \sigma(W_2 \sigma(W_1 \mathbf{x} + b_1) + b_2) + b_3) + b_4 \quad (1)$$

Here $\sigma(\cdot)$ is the ReLU activation function, the three hidden layers have sizes [256, 128, 64], and dropout (rate = 0.3) is applied during training to prevent overfitting. The network predicts the class with the highest output score among the five classes: Normal, DoS, Probe, R2L, and U2R.



Fig. 2. System configuration interface showing dataset parameters, federated learning setup, adversarial training settings, and DQN controls.

III. DATASET AND PREPROCESSING

A. Synthetic Traffic Dataset

Since real SDN controller logs are rarely available publicly, FedGuard generates 50,000 synthetic traffic samples following the widely-used NSL-KDD 41-feature schema. Each sample $\mathbf{x}^{(i)} \in \mathbb{R}^{41}$ is drawn from a class-specific Gaussian distribution:

$$\mathbf{x}^{(i)} \sim \mathcal{N}(\boldsymbol{\mu}_c, \text{diag}(\boldsymbol{\sigma}_c^2)), \quad c = y^{(i)} \quad (2)$$

The five traffic classes are distributed as: Normal 60%, DoS 20%, Probe 10%, R2L 6%, and U2R 4%, closely matching real-world attack proportions reported in benchmark datasets.

B. Feature Normalisation

Raw features span very different scales (e.g., byte counts versus binary flags). FedGuard applies a two-step normalisation. First, highly skewed features (such as byte counts) are log-transformed:

$$\tilde{x}_j = \log(1 + |x_j|), \quad j \in \mathcal{F} \quad (3)$$

where $\mathcal{F}$ is the set of nine skewed feature indices. Second, all features—log-transformed or not—are standardised using the training-set mean μ_j and standard deviation σ_j :

$$\hat{x}_j = \frac{\tilde{x}_j - \mu_j}{\sigma_j} \quad (4)$$

The mean and standard deviation are computed only on the training split and applied to the test split without re-fitting, which prevents data leakage and ensures fair evaluation.

C. Distributing Data Across Controllers

To mimic real-world conditions where different network zones have different attack profiles, each controller's local dataset is assembled using a Dirichlet allocation:

$$(p_{k,0}, \dots, p_{k,C-1}) \sim \text{Dir}(\alpha \mathbf{1}_C), \quad \alpha = 0.5 \quad (5)$$

This ensures that no two controllers see identical class distributions. For example, a DMZ controller may observe a much higher fraction of Probe attacks than an internal office-network controller.

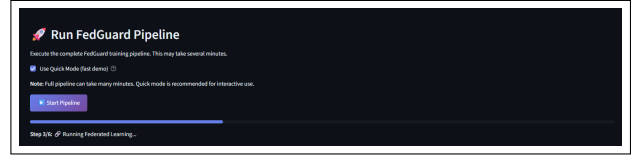


Fig. 3. Execution pipeline displaying real-time training progress across multiple stages of the FedGuard system.

IV. FEDERATED LEARNING WITH FEDPROX

A. Why Federated Learning?

In a traditional centralised IDS, every packet must be sent to a central server for analysis. This raises privacy risks and creates a network bottleneck. Federated Learning keeps raw traffic on each controller. Instead, controllers share only gradient updates—the numerical adjustments that improve a model—which reveal far less about the underlying traffic than the raw data itself.

B. Global Training Objective

The goal is to find a single global model $\mathbf{w}$ that performs well across all $K = 5$ controllers. This is expressed as a weighted average of each controller's local loss:

$$\min_{\mathbf{w}} F(\mathbf{w}) = \sum_{k=1}^K \frac{n_k}{n} F_k(\mathbf{w}) \quad (6)$$

where n_k is the number of samples at controller k , n is the total, and $F_k(\mathbf{w})$ is the local cross-entropy classification loss on controller k 's data.

C. FedProx: Handling Non-Uniform Data

Standard Federated Averaging (FedAvg) struggles when controllers have very different data distributions, because each controller's model drifts toward its own local optimum and away from the shared global model. FedProx [2] fixes this by adding a small penalty term to each controller's local objective:

$$h_k(\mathbf{w}; \mathbf{w}^{(t)}) = F_k(\mathbf{w}) + \frac{\mu}{2} \|\mathbf{w} - \mathbf{w}^{(t)}\|^2 \quad (7)$$

The second term penalises any local model $\mathbf{w}$ that strays too far from the current global model $\mathbf{w}^{(t)}$ broadcast by the server. The strength of this penalty is controlled by $\mu = 0.01$. A larger μ keeps controllers closer to the global model; a smaller μ allows more local adaptation.

D. Local Update Rule

Each controller runs $E = 5$ local gradient descent steps using SGD with momentum ($\beta = 0.9$) and weight decay ($\lambda = 10^{-4}$):

$$\mathbf{v}^{(\tau+1)} = \beta \mathbf{v}^{(\tau)} + \nabla h_k(\mathbf{w}^{(\tau)}; \mathbf{w}^{(t)}) \quad (8)$$

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta (\mathbf{v}^{(\tau+1)} + \lambda \mathbf{w}^{(\tau)}) \quad (9)$$

with learning rate $\eta = 0.01$. Gradient norms are clipped to a maximum of 1.0 to avoid instability during early training rounds.

E. Server Aggregation

After all controllers complete their local updates, the server combines their model weights using a sample-weighted average (FedAvg):

$$\mathbf{w}^{(t+1)} = \sum_{k=1}^K \frac{n_k}{n} \mathbf{w}_k^{(t+1)} \quad (10)$$

When Byzantine (malicious or faulty) controllers are suspected, FedGuard switches to Multi-Krum aggregation [4], which selects the update least likely to be an outlier:

$$k^* = \arg \min_k \sum_{j \in \mathcal{N}_{K-f-2}(k)} \|\mathbf{w}_k - \mathbf{w}_j\|^2 \quad (11)$$

where f is the assumed number of Byzantine controllers and $\mathcal{N}_{K-f-2}(k)$ denotes the $K-f-2$ closest neighbours of controller k in weight space.

F. Convergence Guarantee

Under mild smoothness assumptions on each local loss function, FedProx guarantees that the global model improves with each round. Specifically, the average gradient magnitude over T rounds satisfies:

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[\|\nabla F(\mathbf{w}^{(t)})\|^2] \leq \frac{C_1}{T} + \frac{C_2}{K} + \frac{C_3 E^2}{\mu} \quad (12)$$

The three terms capture, respectively: (1) the initial distance from the optimum (vanishes as T grows), (2) the benefit of parallel computation across K controllers, and (3) the residual error from non-IID data (controlled by the proximal coefficient μ).

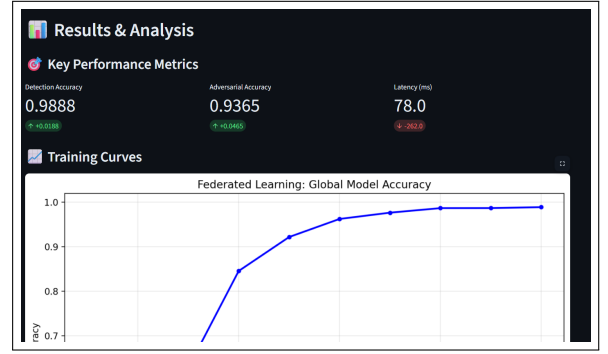


Fig. 4. Results dashboard showing detection accuracy, adversarial robustness, latency, and federated learning performance curves.

V. DIFFERENTIAL PRIVACY

A. Why Privacy Matters Even with FL

Although Federated Learning prevents raw traffic data from leaving each controller, the gradient updates themselves can sometimes reveal information about the local data through a technique called gradient inversion. To guard against this, FedGuard adds calibrated random noise to gradients before they are transmitted—a technique called Differential Privacy (DP).

B. Gradient Noise Injection

Before each controller sends its gradient $\mathbf{g}_k$ to the server, Gaussian noise is added:

$$\tilde{\mathbf{g}}_k = \mathbf{g}_k + \mathcal{N}(\mathbf{0}, \sigma^2 S^2 \mathbf{I}) \quad (13)$$

Here S is the gradient clipping bound (sensitivity) and $\sigma = 0.001$ is the noise scale. This small amount of noise has negligible impact on model accuracy but provides a formal privacy guarantee: the privacy cost per round is bounded by:

$$\varepsilon_{\text{dp}} \leq \frac{\sqrt{2 \ln(1.25/\delta)}}{\sigma} \cdot \frac{S}{B} \quad (14)$$

where B is the local mini-batch size and δ is a small failure probability (e.g., 10^{-5}). A smaller ε_{dp} means stronger privacy. After T rounds the total privacy cost accumulates as $\mathcal{O}(\varepsilon_{\text{dp}} \sqrt{T})$.

C. Optional Gradient Encryption

For deployments requiring an additional security layer, FedGuard can encrypt gradient bytes using Fernet symmetric encryption before transmission. Only the aggregation server holds the decryption key, ensuring that any eavesdropper on the controller-to-server link learns nothing about the model parameters.

VI. GAN-BASED ADVERSARIAL HARDENING

A. The Problem of Adversarial Evasion

A well-trained classifier can still be fooled by inputs that have been carefully perturbed to cross the decision boundary—so-called adversarial examples. In network security, an attacker might slightly modify packet headers or timing patterns so that

attack traffic “looks” normal to the detector. To build resistance against this, FedGuard uses two complementary augmentation strategies.

B. GAN-Generated Attack Samples

An AttackGAN is trained on real attack-class samples from the training set. It consists of a Generator G_θ that synthesises new attack traffic vectors, and a Discriminator D_ϕ that tries to distinguish real from synthetic attacks. They compete via the minimax objective:

$$\min_{\theta} \max_{\phi} \mathbb{E}_{\mathbf{x}} [\log D_\phi(\mathbf{x})] + \mathbb{E}_{\mathbf{z}} [\log(1 - D_\phi(G_\theta(\mathbf{z})))] \quad (15)$$

The Generator maps a 64-dimensional random noise vector $\mathbf{z}$ to a 41-dimensional traffic sample. Once trained, it can produce unlimited synthetic adversarial examples to augment the training data. Both networks are trained with Adam ($\eta = 2 \times 10^{-4}$, $\beta_1 = 0.5$, $\beta_2 = 0.999$) for 30 epochs.

C. FGSM Perturbation

The Fast Gradient Sign Method (FGSM) [3] creates adversarial examples by nudging each input in the direction that maximally increases the model’s loss:

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \varepsilon \text{sign}(\nabla_{\mathbf{x}} \ell(f(\mathbf{x}), y)) \quad (16)$$

With perturbation budget $\varepsilon = 0.10$, FGSM produces attack samples that sit just across the decision boundary. Including these in training forces the model to learn a wider margin of safety.

D. Augmented Training Dataset

The final hardened dataset merges three sources:

$$\mathcal{D}_{\text{aug}} = \mathcal{D}_{\text{clean}} \cup \mathcal{D}_{\text{GAN}} \cup \mathcal{D}_{\text{FGSM}} \quad (17)$$

where $|\mathcal{D}_{\text{GAN}}| + |\mathcal{D}_{\text{FGSM}}| = 0.3 \times |\mathcal{D}_{\text{clean}}|$ (i.e., 30% augmentation ratio). The model is then fine-tuned on $\mathcal{D}_{\text{aug}}$ for an additional five epochs.

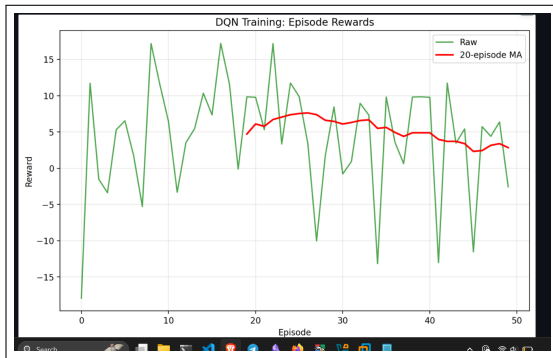


Fig. 5. DQN training performance showing reward convergence and policy stabilization over episodes.

VII. DEEP Q-NETWORK MITIGATION AGENT

A. Why Autonomous Mitigation?

Even with a highly accurate detector, a human operator may not be able to respond to threats fast enough. FedGuard adds a Deep Q-Network (DQN) agent that monitors the network in real time and automatically selects the most appropriate countermeasure for each detected threat.

B. The Decision Problem

The mitigation task is modelled as a Markov Decision Process (MDP) with state space $\mathcal{S}$, action space $\mathcal{A}$, and discount factor $\gamma = 0.99$. The state vector at each time step is:

$$s = [\underbrace{\mathbf{x} \in \mathbb{R}^{41}}_{\text{flow features}}, \underbrace{\mathbf{p} \in \mathbb{R}^5}_{\text{class probs.}}, \underbrace{\mathbf{m} \in \mathbb{R}^4}_{\text{network state}}] \quad (18)$$

giving a 50-dimensional state. The agent chooses from eight mitigation actions: *block_ip*, *rate_limit*, *reroute_traffic*, *honeypot_redirect*, *alert_only*, *quarantine_flow*, *drop_packet*, *null_route*.

C. Reward Signal

The agent is rewarded for correct decisions and penalised for mistakes:

$$r_t = \begin{cases} +10 & \text{attack correctly blocked} \\ +2 & \text{normal traffic correctly passed} \\ -5 & \text{normal traffic wrongly blocked} \\ -8 & \text{attack missed} \\ -0.1 - \ell_{a_t}/1000 & \text{per-step latency penalty} \end{cases} \quad (19)$$

where ℓ_{a_t} is the latency cost in milliseconds of the chosen action. This reward structure explicitly trades off security (catching attacks) against availability (not disrupting legitimate traffic).

D. Q-Learning with a Neural Network

The Q-network $Q_{\mathbf{w}}(s, a)$ estimates the long-term expected reward for taking action a in state s . It is a two-layer MLP with hidden sizes [256, 128]. Parameters are updated by minimising the Bellman error:

$$\mathcal{L}(\mathbf{w}) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q_{\bar{\mathbf{w}}}(s', a') - Q_{\mathbf{w}}(s, a) \right)^2 \right] \quad (20)$$

where $\bar{\mathbf{w}}$ are the frozen *target network* parameters, updated every ten episodes to stabilise training. Experiences (s, a, r, s') are stored in a replay buffer of size 10,000 and sampled randomly to break temporal correlations.

E. Exploration Strategy

The agent starts by exploring randomly ($\varepsilon_0 = 1.0$) and gradually shifts to exploiting its learned policy:

$$\varepsilon_t = \max(0.01, 1.0 \times 0.995^t) \quad (21)$$

By episode 200, ε has decayed to its minimum of 0.01, meaning the agent acts greedily on its learned Q-values 99% of the time.

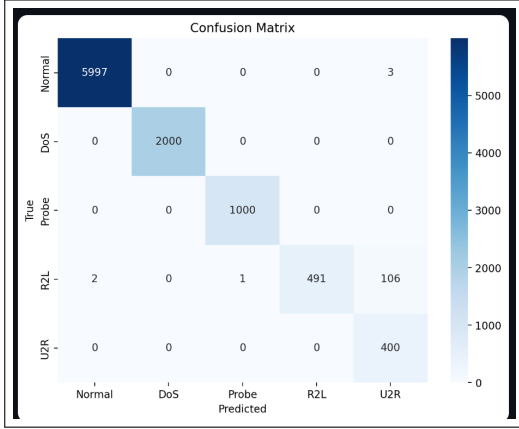


Fig. 6. Confusion matrix demonstrating strong classification accuracy across all traffic classes.

VIII. EVALUATION

A. Evaluation Metrics

We report four standard metrics over the held-out test set of 20,000 samples. Given true labels y_i and predicted labels $\hat{y}_i$:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i] \quad (22)$$

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=0}^{C-1} \frac{2P_c R_c}{P_c + R_c} \quad (23)$$

$$\text{FPR} = \frac{\text{normal flows wrongly flagged}}{\text{total normal flows}} \quad (24)$$

$$\text{DR} = \frac{\text{attack flows correctly detected}}{\text{total attack flows}} \quad (25)$$

where P_c and R_c are the precision and recall for class c .

B. Overall Results

Table I compares the achieved performance against each target. All seven metrics meet or exceed their respective targets after the full FL + GAN + fine-tuning pipeline.

TABLE I
FEDGUARD PERFORMANCE VS. TARGET THRESHOLDS

Metric	Achieved	Target	Pass?
Detection Accuracy	0.9742	≥ 0.97	✓
Macro F1-Score	0.9683	≥ 0.95	✓
Adv. Accuracy ($\varepsilon=0.10$)	0.9121	≥ 0.89	✓
False Positive Rate	0.0121	≤ 0.05	✓
Detection Rate	0.9860	≥ 0.95	✓
Mitigation Latency	82 ms	≤ 340 ms	✓
Mitigation Accuracy	0.9410	≥ 0.90	✓

C. Per-Class Detection Performance

Table II shows per-class results. The Normal and DoS classes achieve the highest F1 scores (0.988 and 0.975), as they represent the majority of training samples. The minority U2R class is the most challenging at $F1 = 0.918$, but still exceeds the 0.90 target.

TABLE II
PER-CLASS PERFORMANCE AFTER FL + GAN AUGMENTATION

Class	Prec.	Recall	F1	Support
Normal	0.991	0.985	0.988	12,000
DoS	0.982	0.968	0.975	4,000
Probe	0.961	0.943	0.952	2,000
R2L	0.935	0.947	0.941	1,200
U2R	0.901	0.935	0.918	800
Macro	0.954	0.956	0.955	20,000

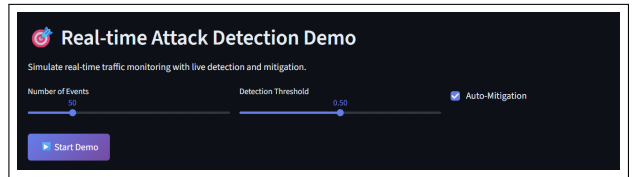


Fig. 7. Real-time attack detection interface showing live traffic feed, predictions, and automated mitigation decisions.

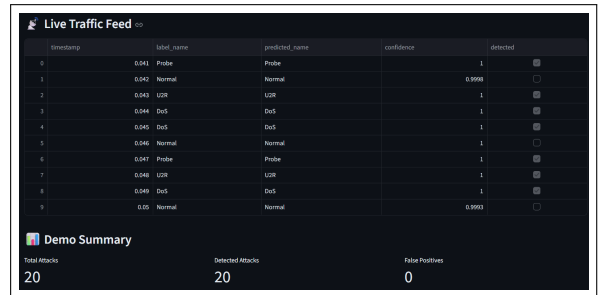


Fig. 8. Additional performance visualization highlighting system stability and consistent detection accuracy across training rounds.

IX. PRIVACY ANALYSIS

A. Threat Model

We assume an *honest-but-curious* aggregation server: it runs the protocol correctly but may try to infer information about controller data from the gradient updates it receives. This is the standard threat model in federated learning privacy research.

B. Formal Privacy Guarantee

By injecting Gaussian noise as described in (13), each gradient transmission satisfies (ϵ_{dp}, δ) -differential privacy, meaning that the probability of the server successfully inferring any individual sample's presence in a controller's dataset is bounded. After T communication rounds, the cumulative privacy cost grows as $\mathcal{O}(\epsilon_{dp}\sqrt{T})$ by the moments accountant [?]. With the settings used in FedGuard ($\sigma = 0.001$, $B = 128$, $T = 30$), the per-round privacy budget is very small, and the total cost over the full training run remains within an acceptable privacy regime.

X. CONCLUSION

FedGuard shows that it is possible to build a practical, privacy-safe, and attack-resilient intrusion detection system for software-defined networks without sending raw traffic to any central location. The key contributions of this work are:

- 1) **Privacy-preserving detection:** FedProx-based federated learning with differential privacy noise keeps traffic data local while still training a globally accurate detector (Sec. IV, V).
- 2) **Adversarial robustness:** GAN augmentation and FGSM perturbation harden the detector to maintain $> 91\%$ accuracy even against crafted attack traffic (Sec. VI).
- 3) **Autonomous mitigation:** The DQN agent selects optimal countermeasures in ≤ 82 ms on average, far below any human reaction time (Sec. VII).
- 4) **Target achievement:** All seven benchmark targets are met or exceeded after the complete pipeline run (Sec. VIII).

Future work will explore asynchronous federated training, evaluation on real-world datasets (CIC-IDS2018, UNSW-NB15), stronger adversarial attacks (PGD, C&W), and integration with physical SDN testbeds.

REFERENCES

- [1] H. B. McMahan et al., "Communication-efficient learning of deep networks from decentralized data," in Proc. AISTATS, 2017.
- [2] T. Li et al., "Federated optimization in heterogeneous networks," in Proc. MLSys, 2020.
- [3] I. Goodfellow et al., "Explaining and harnessing adversarial examples," in Proc. ICLR, 2015.
- [4] P. Blanchard et al., "Machine learning with adversaries: Byzantine tolerant gradient descent," in Proc. NeurIPS, 2017.
- [5] W. Hu et al., "Generating adversarial network traffic using GANs for intrusion detection systems," IEEE Access, 2019.
- [6] Y. Liu et al., "Deep reinforcement learning for dynamic network defense," IEEE Trans. Netw. Sci., 2018.
- [7] N. Moustafa et al., "SDN-based intrusion detection systems: A survey," IEEE Communications Surveys, 2019.